Markov Decision Processes: the Navigation Problem

Khalil BIBI
TIPE (research project, preparatory classes) · Lycée Louis-le-Grand

Problem:



Source: Pexels

Goal: model, then search for optimal paths around obstacles in water currents

Outline

1. Context of the problem and modelling with Markov decision processes
2. Value iteration algorithm and the Bellman operator
3. The exploration/exploitation dilemma, applied in practice with SARSA and Q-learning

1.1: Definition of an MDP

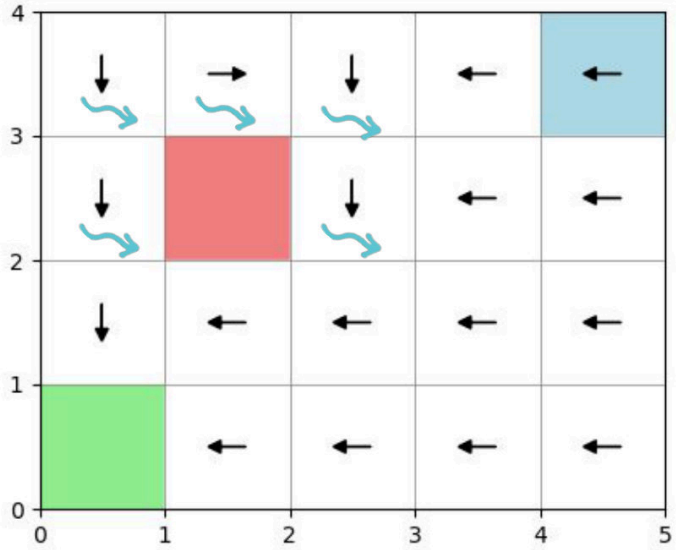
- $\mathcal{S}$, a finite set of states.
- $\mathcal{A}$, a finite set of actions.
- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$, the transition probability function.
- $R : \mathcal{S} \to \mathbb{R}$, the reward function.
- $\gamma \in [0, 1]$, the discount factor.

Definition of a deterministic policy:

- $\pi : \mathcal{S} \to \mathcal{A}$, a deterministic function that maps each state to an action.

1.2: Modelling the problem

Example of a board, currents and policy:



Modelling the water currents:

```
#Convention: North, East, West, South
pba = [ [0.9, 0.1, 0.0, 0.0],
        [0.0, 0.9, 0.0, 0.1],
        [0.0, 0.1, 0.8, 0.1],
        [0.0, 0.2, 0.0, 0.8]]
```

For example: pba[0][0] gives the probability of ending up north given that we want to go north

1.3: State-action function (Q) and state-value function (V)

$$Q^{\pi}(s,a) = \mathbb{E}^{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}R(s_t)\,\middle|\, s_0 = s,\, a_0 = a\right]$$

$$V^{\pi}(s) = \mathbb{E}^{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}R(s_t)\,\middle|\, s_0 = s\right]$$

2.1: Bellman equation

$$V^{\pi}(s) = \sum_{s' \in \mathcal{S}} P(s, \pi(s), s') \left[R(s') + \gamma V^{\pi}(s') \right]$$

$$Q^{\pi}(s, a) = \sum_{s' \in \mathcal{S}} P(s, a, s') \left[R(s') + \gamma V^{\pi}(s') \right]$$

2.1: Optimal policy

The Bellman operator is defined as:

$$\mathcal{B}(V)(s) = \max_{a \in \mathcal{A}} \left[\sum_{s' \in \mathcal{S}} P(s, a, s') \left[R(s') + \gamma V(s') \right] \right]$$

and the optimal value V^* is defined by:

$$V^*(s) = \max_{\pi} \left[V^{\pi}(s) \right]$$

In particular, it is the unique fixed point of the operator: $\mathcal{B}(V^*) = V^*$

2.2: Value iteration

repeat

$\Delta \leftarrow 0$

for all $s \in \mathcal{S}$ **do**

$v \leftarrow V(s)$

$V(s) \leftarrow \max_{a \in \mathcal{A}} \left(\sum_{s' \in \mathcal{S}} P(s, a, s') \left[R(s') + \gamma V(s') \right] \right)$

▷ Equivalent to $V(s) \leftarrow \max_{a \in \mathcal{A}} Q(s, a)$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

end for

until $\Delta < \theta$ ▷ Convergence criterion

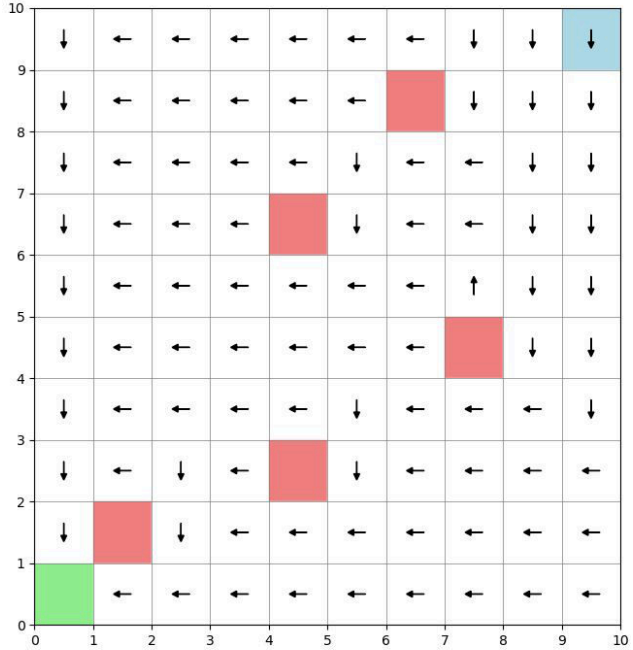
2.2: Policy extraction

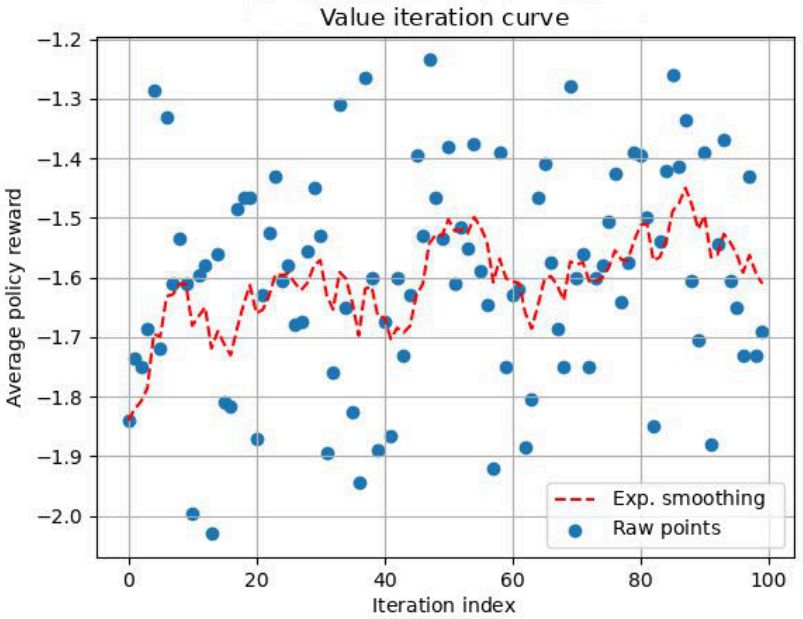
$a_{\text{optimal}} \leftarrow \arg\max_{a \in \mathcal{A}} \left(\sum_{s' \in \mathcal{S}} P(s, a, s') \left[R(s') + \gamma V(s') \right] \right)$

$\pi(s) \leftarrow a_{\text{optimal}}$

▷ Equivalent to $a_{\text{optimal}} \leftarrow \arg\max_{a \in \mathcal{A}} Q(s, a)$

2.3: Performance:





3.1: The exploitation/exploration problem

Draw a random number p between 0 and 1

if $p < \varepsilon$ **then**

    Choose a random action a

else

    Choose the action a that maximises $Q(s, a)$

Variants and other methods:

- Decrease epsilon at each round (more stable in the long run)
- UCB1 (converges very quickly, then stabilises)

3.2: General temporal-difference method

Algorithm 2 TD Method

1: Start from an initial state

2: $s \leftarrow$ initial state

3: **while** s is not a final state **do**

4: Choose an action a with ε-greedy

5: Execute a and observe the new state s'

6: Update $Q(s, a)$:

$$Q(s,a) \leftarrow (1-\alpha)\, Q(s,a) + \alpha\,(r + \gamma \cdot \text{estimate of the future gain})$$

7: $s \leftarrow s'$

8: **end while**

3.3: Q-Learning

Select A using a policy based on Q (for example, epsilon-greedy)

Take action A, observe R (reward), S' (new state)

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_a Q(S', a) - Q(S, A) \right]$$

3.4: SARSA

Take action A, observe R, S'

Select A' using a policy based on Q (for example, epsilon-greedy)

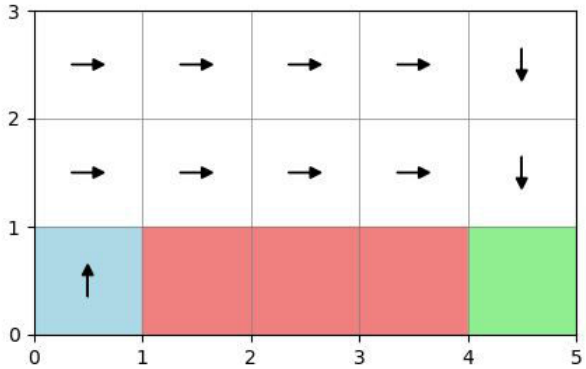
$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma Q(S', A') - Q(S, A) \right]$$

$$S \leftarrow S'$$

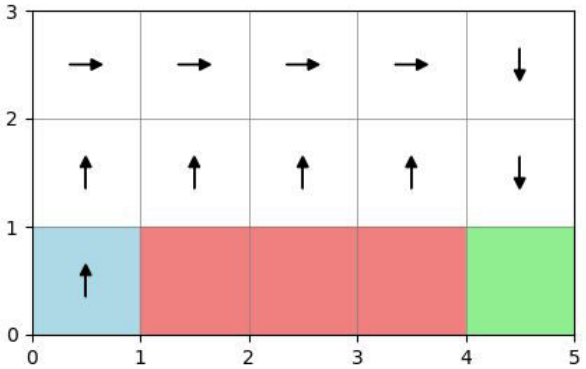
$$A \leftarrow A'$$

3.5: Comparison of behaviours

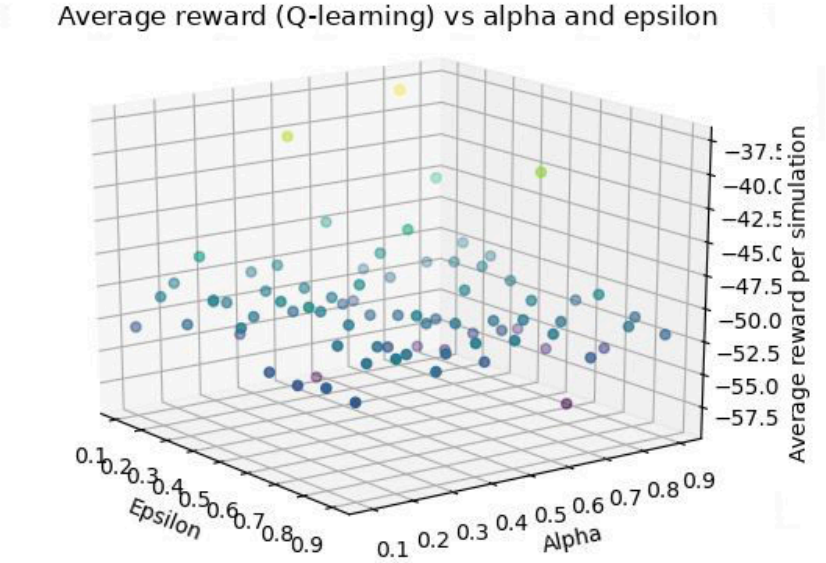
Q-Learning: optimism



SARSA: risk minimisation

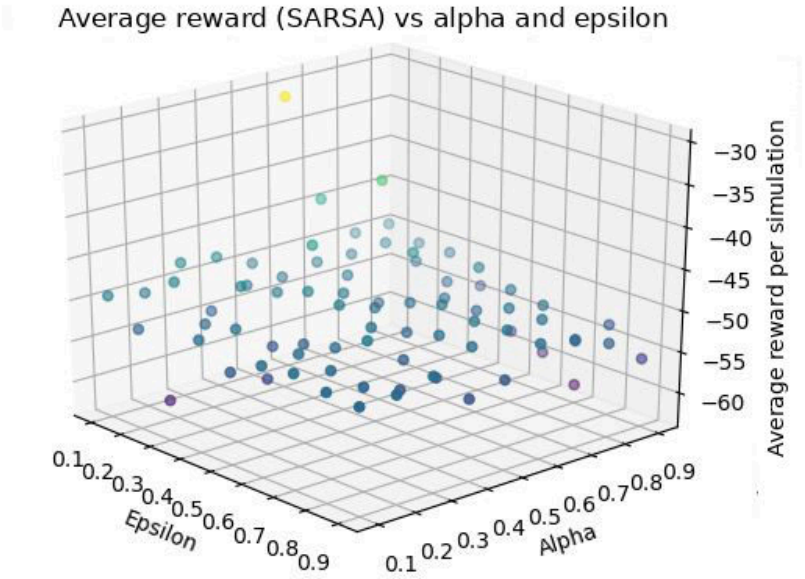


3.6: Q-Learning: dependence on epsilon and on the learning rate



Optimal epsilon and alpha (for a randomly drawn 10 × 10 board with 5 obstacles):
epsilon = 0.1
alpha = 0.8

3.6: SARSA: dependence on epsilon and on the learning rate



Optimal epsilon and alpha (for a randomly drawn 10 × 10 board with 5 obstacles):
epsilon = 0.1
alpha = 0.8

Conclusion:

- Ideal approach: policy iteration
- Realistic approach:
  a. SARSA: minimises risk
  b. Q-Learning: optimism about the gain

Appendix

Appendix 1: The multi-armed bandit problem

Algorithm 1 Epsilon-greedy for the multi-armed bandit problem

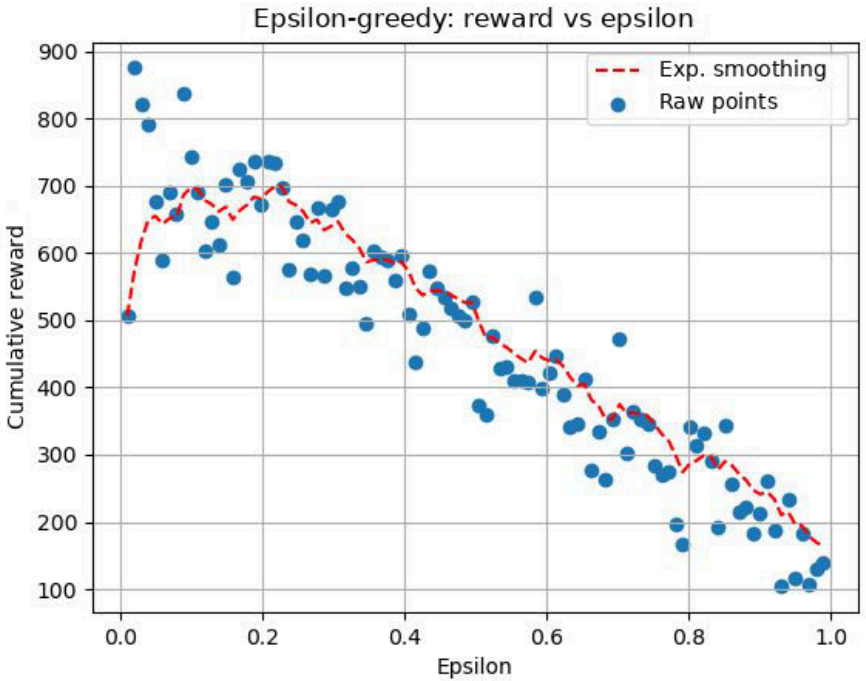
Initialise the action values $Q(a)$ and the number of times each action is selected $N(a)$, for each action a
Initialise the action set A and the exploration parameter ε
$t \leftarrow 0$
while True **do**
    Draw a random number $x \in [0, 1]$
    if $x < \varepsilon$ **then**
        Select a random action $a_t \in A$
    else
        Select $a_t = \arg\max_{a \in A} Q(a)$
    end if
    Execute a_t, observe the reward r_t
    Update the action estimates:

$$N(a_t) \leftarrow N(a_t) + 1$$
$$Q(a_t) \leftarrow Q(a_t) + (r_t - Q(a_t)) / N(a_t)$$

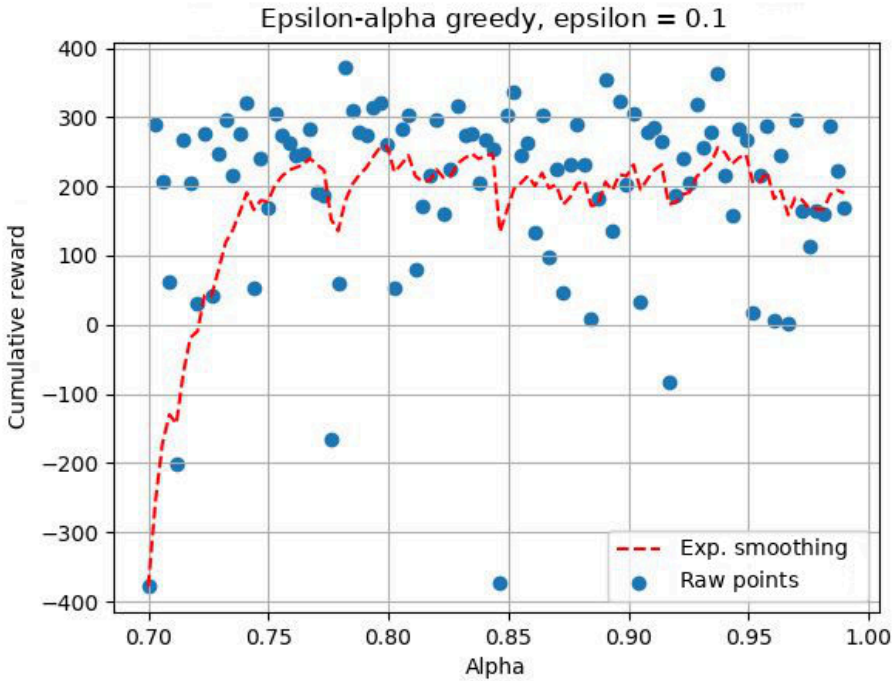
    $t \leftarrow t + 1$
end while

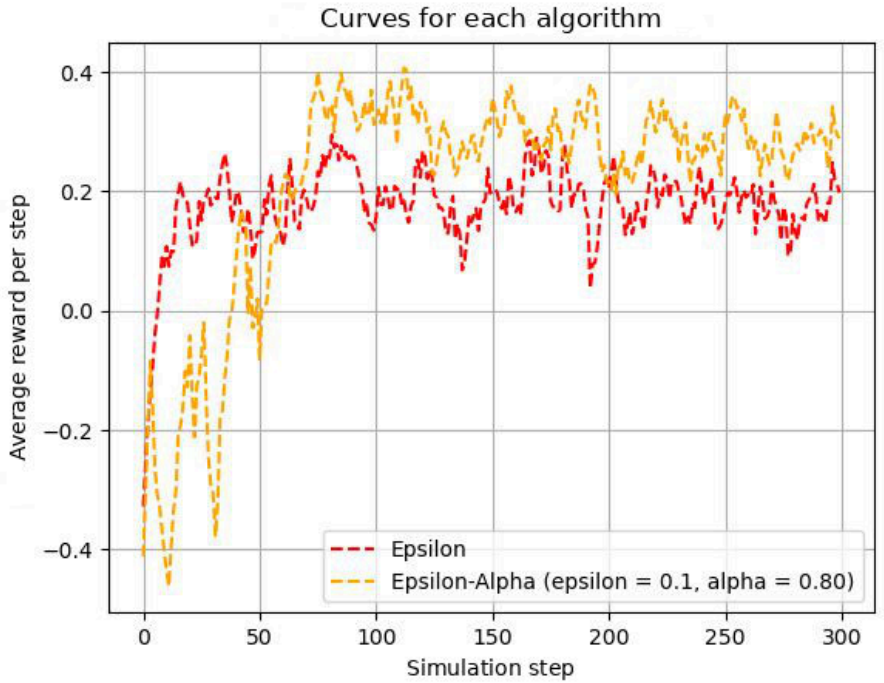
Values used for the tests:

```
rec = [1.1 for i in range(10)]
pba = [0.3,0.6,0.2,0.3,0.5,
       0.2,0.1,0.4,0.62,0.58]
```


Epsilon-greedy: reward vs epsilon
Exp. smoothing
Raw points
Cumulative reward
Epsilon
900
800
700
600
500
400
300
200
100
0.0
0.2
0.4
0.6
0.8
1.0

Epsilon-alpha curve:




Curves for each algorithm
Average reward per step
Simulation step
0.4
0.2
0.0
−0.2
−0.4
0
50
100
150
200
250
300
Epsilon
Epsilon-Alpha (epsilon = 0.1, alpha = 0.80)

Appendix 3: Code

The code is shown as written, with French identifiers:
`etats` states · `taille` size · `pos_dep` start position · `obs` obstacles · `rec` rewards · `hauteur` / `largeur` height / width · `nord`, `sud`, `est`, `ouest` north, south, east, west · `verifier_pos` check position · `simuler` simulate · `jouer` play · `actions_possibles` possible actions · `valeurs` values

```
class maze:

    def __init__(self,taille,pos_dep,obs,rec,pba,gamma):
        etats = []
        for i in range(taille[0]):
            lst = [-0.5 for k in range(taille[1])]
            etats.append(lst)
        for pos in obs:
            etats[pos[0]][pos[1]] = -pos[2]
        for pos in rec:
            etats[pos[0]][pos[1]] = pos[2]

        self.args = (taille,pos_dep,obs,rec,pba,gamma)
        self.x = pos_dep[1]
        self.y = pos_dep[0]
        self.etats = etats
        self.end = False
        self.gamma = gamma
        self.cumu_reward = 0.0

        self.hauteur = taille[0]
        self.largeur = taille[1]

        self.NN = pba[0][0]
        self.NE = pba[0][1]
        self.NO = pba[0][2]
        self.NS = pba[0][3]

        self.EN = pba[1][0]
        self.EE = pba[1][1]
        self.EO = pba[1][2]
        self.ES = pba[1][3]

        self.ON = pba[2][0]
        self.OE = pba[2][1]
        self.OO = pba[2][2]
        self.OS = pba[2][3]
```

```
def verifier_pos(self):
    if self.x < 0:
        self.x = 0
    if self.y < 0:
        self.y = 0
    if self.y >= self.hauteur:
        self.y = self.hauteur-1
    if self.x >= self.largeur:
        self.x = self.largeur-1

    if self.etats[self.y][self.x] > 0:
        self.end = True


def nord(self):
    p = uniform(0,1)
    self.cumu_reward += self.etats[self.y][self.x]
    if p < self.NN:
        self.y -= 1
    elif p < self.NN + self.NE:
        self.x += 1
    elif p < self.NN + self.NE + self.NO:
        self.x -= 1
    else:
        self.y += 1
    self.verifier_pos()
```

```
def __init__(self, lab, valeurs):
    self.lab = lab
    self.values = valeurs
    lst = [[] for i in range (lab.hauteur)]
    for i in range(lab.hauteur):
        lst[i] = ["Nord" for h in range(lab.largeur)]
    self.policy = lst


def q_value(self,a,x,y):
    new_value = 0.0

    if a == "Nord":
        if y == 0:
            new_value += self.lab.NN * (self.lab.etats[y][x] + self.lab.gamma*self.values[y][x])
        else:
            new_value += self.lab.NN * (self.lab.etats[y-1][x] + self.lab.gamma*self.values[y-1][x])
        if y == (self.lab.hauteur-1):
            new_value += self.lab.NS *(self.lab.etats[y][x] + self.lab.gamma * self.values[y][x])
        else:
            new_value += self.lab.NS * (self.lab.etats[y+1][x] + self.lab.gamma * self.values[y+1][x])
        if x == 0:
            new_value += self.lab.NO*(self.lab.etats[y][x]+ self.lab.gamma*self.values[y][x])
        else:
            new_value += self.lab.NO*(self.lab.etats[y][x-1] + self.lab.gamma*self.values[y][x-1])

        if x == (self.lab.largeur - 1):
            new_value += self.lab.NE*(self.lab.etats[y][x] + self.lab.gamma*(self.values[y][x]))
        else:
            new_value += self.lab.NE*(self.lab.etats[y][x+1] + self.lab.gamma*(self.values[y][x+1]))
    elif a == "Sud":
```

```
def update_values(self,theta, iter_max=100):
    if iter_max > 0:
        delta = 0
        for x in range(self.lab.largeur):
            for y in range(self.lab.hauteur):
                actions = self.lab.actions_possibles(x,y)
                q = []
                for a in actions:
                    new_value = self.q_value(a,x,y)
                    q.append((a, new_value))
                m_v = q[0]
                for i in range(1, len(q)):
                    if q[i][1] > m_v[1]:
                        m_v = q[i]

                delta = max(delta, abs(m_v[1] - self.values[y][x]))
                self.values[y][x] = m_v[1]
                self.policy[y][x] = m_v[0]
        if delta > theta:
            self.update_values(theta, iter_max-1)
```

```
def simuler(self, iter_max = 100):
    i = iter_max
    while self.lab.end != True and i > 0:
        action = self.policy[self.lab.y][self.lab.x]
        if action == "Nord":
            self.lab.nord()
        elif action == "Sud":
            self.lab.sud()
        elif action == "Ouest":
            self.lab.ouest()
        else:
            self.lab.est()
        i -= 1
         print(self.lab.x, self.lab.y, self.lab.cumu_reward)
         self.visualize_values()

    self.lab.cumu_reward += self.lab.etats[self.lab.y][self.lab.x]
    return self.lab.cumu_reward

def eva_algo(self,n,theta):
    lst = []
    x0 = self.lab.x
    y0 = self.lab.y
    for i in range(n):
        self.update_values(theta, 1)
        rc = 0
        for k in range(100):
            rc += self.simuler()
            self.lab.x = x0
            self.lab.y = y0
            self.lab.cumu_reward = 0.0
            self.lab.end = False
        rc = rc/100
        lst.append(rc)
```

```
class temp_d():

    def __init__(self, learn_rate, maze, epsilon):
        lst = []
        lst2 = []
        pol = []
        for y in range(maze.hauteur):
            nv_lst1 = []
            nv_lst2 = []
            for x in range(maze.largeur):
                nv_dict_keys = maze.actions_possibles(x,y)
                dc1 = {}
                dc2 = {}
                for k in nv_dict_keys:
                    dc1[k] = 0
                    dc2[k] = 0
                nv_lst1.append(dc1)
                nv_lst2.append(dc2)
            lst.append(nv_lst1)
            lst2.append(nv_lst2)
            pol.append([ "Nord" for j in range(maze.largeur)])

        self.maze = maze
        self.learn_rate = learn_rate
        self.q_values = lst
        self.epsi = epsilon
        self.n_actions = lst2
        self.policy = pol
```

```
def sarsa(self,theta,n_iter = 100):
    if n_iter > 0:
        x0 = self.maze.x
        y0 = self.maze.y
        an_Valeurs = copy.deepcopy(self.q_values)
        delta_Q = 0
        while self.maze.end == False:
            action = self.e_greedy()
            pos_y = self.maze.y
            pos_x = self.maze.x
            r_curr = self.maze.cumu_reward

            self.jouer(action)
            r = self.maze.cumu_reward - r_curr
            action2 = self.e_greedy()
            pos_y2 = self.maze.y
            pos_x2 = self.maze.x

            delta = r + self.maze.gamma*self.q_values[pos_y2][pos_x2][action2] - self.q_values[pos_y][pos_x][action]
            self.q_values[pos_y][pos_x][action] += self.learn_rate*delta

        self.maze.end = False
        self.maze.x = x0
        self.maze.y = y0
        self.maze.cumu_reward = 0

        for y in range(self.maze.hauteur):
            for x in range(self.maze.largeur):
                for i in list(self.q_values[y][x].keys()):
                    diff = abs(self.q_values[y][x][i] - an_Valeurs[y][x][i])
                    if diff > delta_Q:
                        delta_Q = diff

        if delta_Q > theta or n_iter-1 > 0:
            self.q_learning_eps(theta, n_iter-1)
```

```
def policy_extract(self):
    for y in range(self.maze.hauteur):
        for x in range(self.maze.largeur):
            act_poss = list(self.q_values[y][x].keys())
            i0 = act_poss[0]
            for i in act_poss:
                if self.q_values[y][x][i] > self.q_values[y][x][i0]:
                    i0 = i
            self.policy[y][x] = i0
```